Software Requirements Document

Color Clash - A Two-Player Fighting Game

TOBB ETU – BIL 482

February 2026

Team Members

Mehmet Umur ÖZÜ

Mehmet Yasin TOSUN

Mustafa GÖZÜTOK

Table of Contents

0. Definitions, Acronyms, Abbreviations, and References

0.1 Definitions

Term	Definition
Archetype	A predefined playable character template with its own initial attributes and compatible elemental mode set.
Elemental Mode	A runtime combat style that modifies attack, defense, mobility, and visual behavior.
Hitbox	The active collision region generated during an attack to determine whether an opponent is struck.
Hurtbox	The collision region representing the vulnerable area of a character that can receive damage.
Tournament Bracket	The elimination structure used to progress competitors through sequential rounds until final ranking is determined.
Object Pool	A reuse mechanism for temporary objects such as particles and hitboxes to reduce allocation overhead during gameplay.

0.2 Acronyms and Abbreviations

Abbreviation	Meaning
AABB	Axis-Aligned Bounding Box
API	Application Programming Interface
DOM	Document Object Model
FPS	Frames Per Second
HUD	Heads-Up Display
MVP	Minimum Viable Product
NFR	Non-Functional Requirement
SDD	Software Design Document
UC	Use Case
UI	User Interface
UML	Unified Modeling Language

0.3 References

Ref. No	Reference
[R1]	Color Clash Project Repository
[R2]	Software Requirements Specification (SRS)
[R3]	Software Design Document
[R4]	Delta Design & Implementation Report
[R5]	Color Clash Test Report

1. System Overview

Color Clash is a web-based 2D fighting game developed using TypeScript and HTML5 Canvas. Unlike traditional high-violence fighting games, this project emphasizes a vibrant, child-friendly aesthetic and strategic depth. It serves as a technical case study for implementing advanced software design patterns in a real-time game engine environment.

2. System Context

The application is a standalone, client-side web application.

- Platform: Desktop/Laptop web browsers (Chrome, Firefox, Safari).
- Technology Stack: TypeScript (Logic), HTML5 Canvas (Rendering), Vite (Build System).
- External Interfaces: Browser DOM for UI and the Keyboard API for user input.

3. Key Features and Functionality

- UC1 - Start Match or Tournament: Initialization of the singleton game engine and setup of the core game state. Players can choose to start a classic 1v1 duel or initiate a "Tournament Mode" for up to 8 competitors. The system handles character instantiation dynamically via a Factory and sets up input listeners based on active matches.
- UC2 - Execute Combat Actions: Real-time character control including movement (left, right, jump), accurate AABB collision resolution between players, and hitbox-based combat processing (attacks, hit stun, and knockback).
- UC3 - Dynamic Elemental Strategy (6 Modes): Implementation of the Strategy pattern allowing characters to toggle between six distinct elemental modes at runtime, affecting combat attributes, visuals, and movement physics dynamically.
- UC4 - Manage Tournament Bracket: Automated handling of tournament brackets. The system pairs competitors, tracks match results, advances winners to subsequent rounds (including automatic byes if necessary), displays round progress animations, and finally reveals a championship podium for the top 3 players.

Available Elemental Modes

- Fire: Aggressive form with high damage multiplier.
- Water: Defensive form focusing on high defense and fluid combat.
- Earth: Armored form providing the highest defense at the cost of mobility.
- Wind: Aerial form that reduces gravity and enables flight capabilities.
- Light: Ranged form specializing in attack range multipliers and beam visuals.
- Dark: Absorption form that converts a portion of incoming damage into bonus attack power.

4. Assumptions and Dependencies

• Assumptions: Players interact via a single shared keyboard for local multiplayer.

• Dependencies: Requires Node.js for development, TypeScript for compilation, and Vite for the development server and bundling.

4.1 Operational Constraints

• The project is designed for local multiplayer on a shared keyboard.

• The application is intended for desktop/laptop browsers and is not optimized for mobile devices.

• The delivered MVP excludes networking features due to synchronization complexity and scope control.

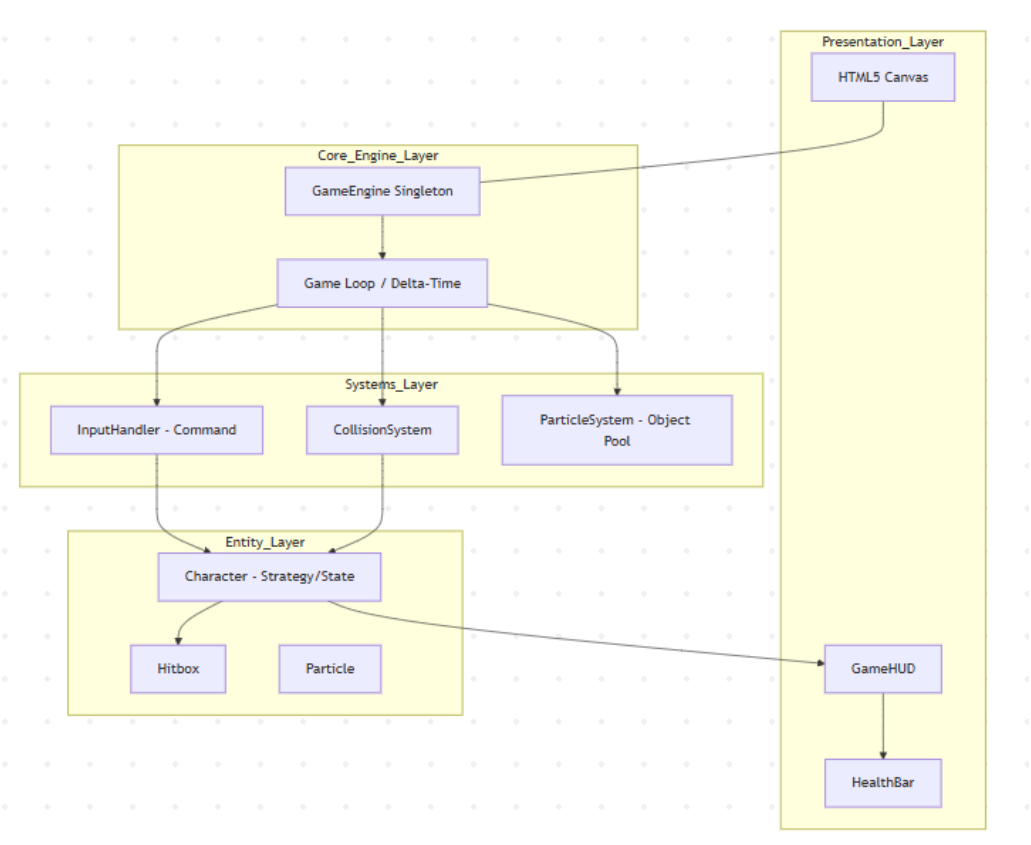
• The game is executed as a client-side browser application and does not require a dedicated backend service.

• Production delivery is based on static bundled files generated by Vite.

5. Architectural Design

5.1 System Architecture Diagram

The system follows a Layered Architecture to decouple game logic from the rendering hardware.



5.2 Architectural Patterns and Styles

- Singleton Pattern: Applied to the GameEngine to ensure a single authoritative source for the game loop and state management.
- Layered Style: Separates core combat physics from the UI/Rendering logic, facilitating independent testing.

5.3 Rationale for Architectural Decisions

The Singleton ensures that multiple engine instances do not compete for the CPU or Canvas context. The layered architecture allows for future-proofing; for instance, the rendering layer could be swapped for WebGL without modifying the core physics logic.

6. Component Design

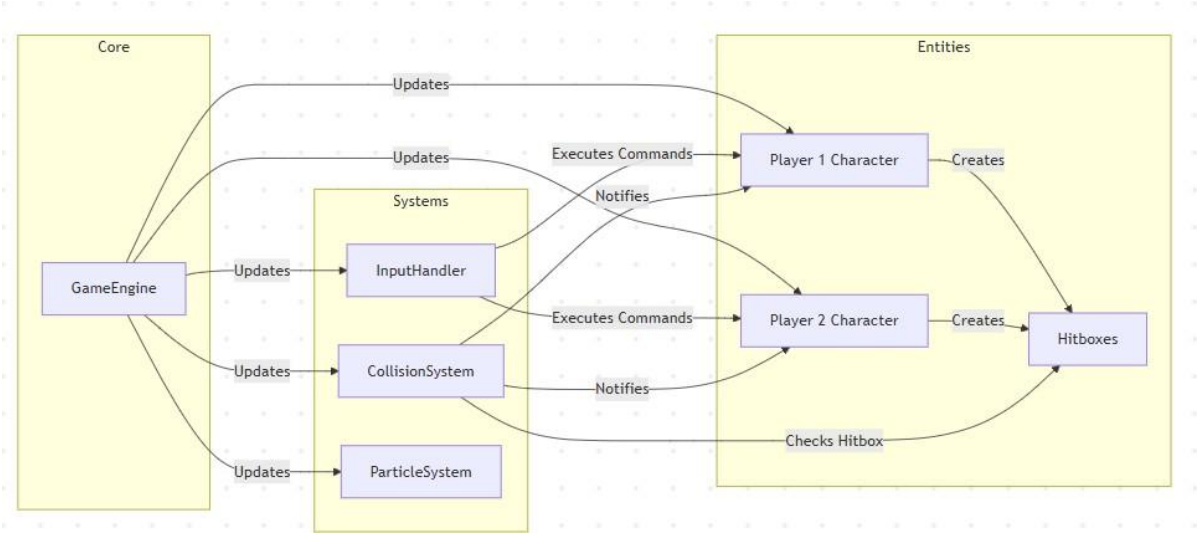
6.1 Subsystems and Modules

- GameEngine: Orchestrates the initialization and the main requestAnimationFrame loop.
- InputHandler: Maps raw browser keyboard events to specific game commands.
- CollisionSystem: Handles AABB logic to prevent overlaps and detect hits.
- CharacterFactory: Centralized module for creating player entities with specific configurations.

6.2 Responsibilities of Each Component

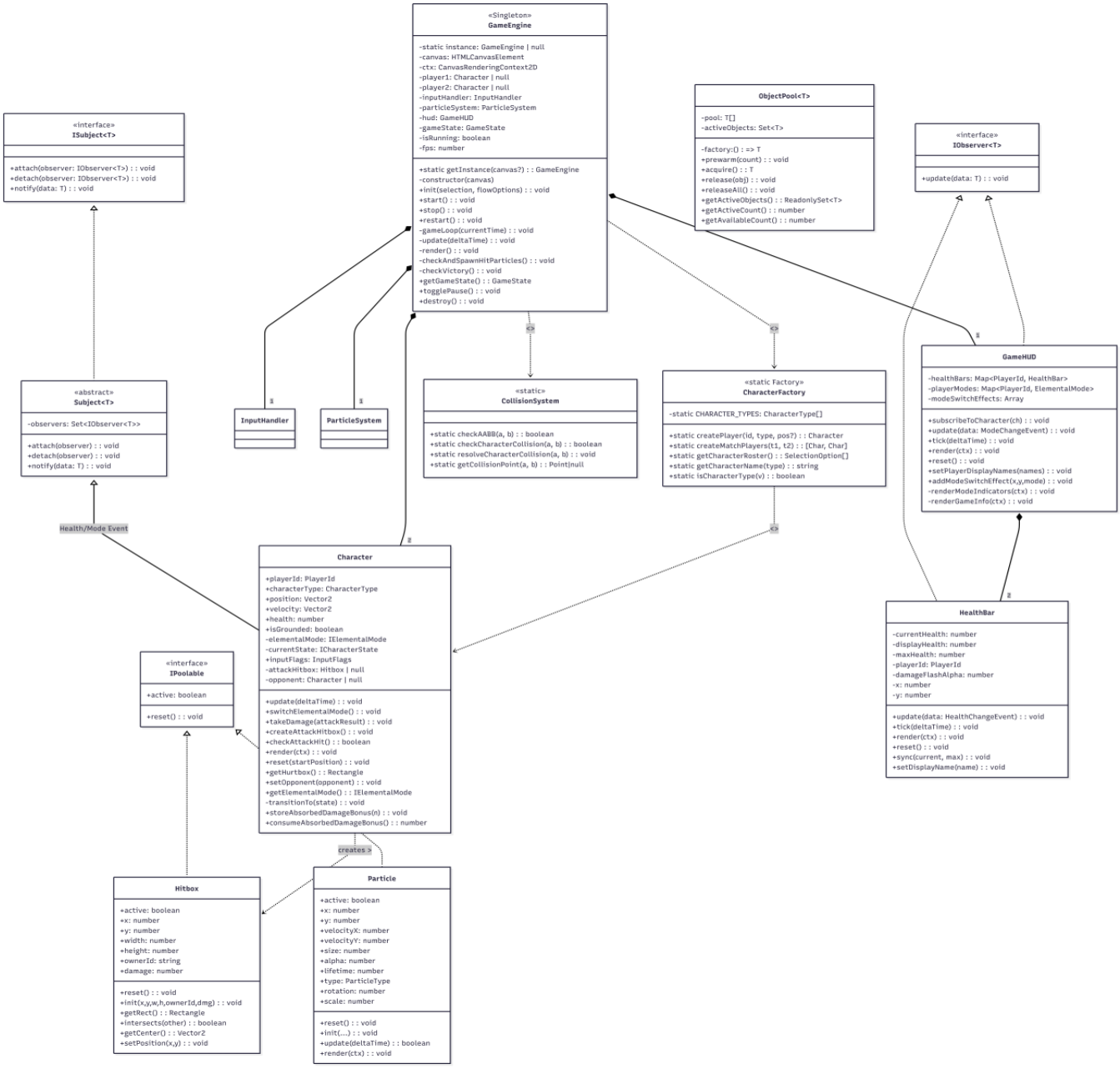
- Character: Manages individual health, position, velocity, and current state.
- Physics Engine: Applies gravity and friction constants to active entities.

6.3 Component Diagrams



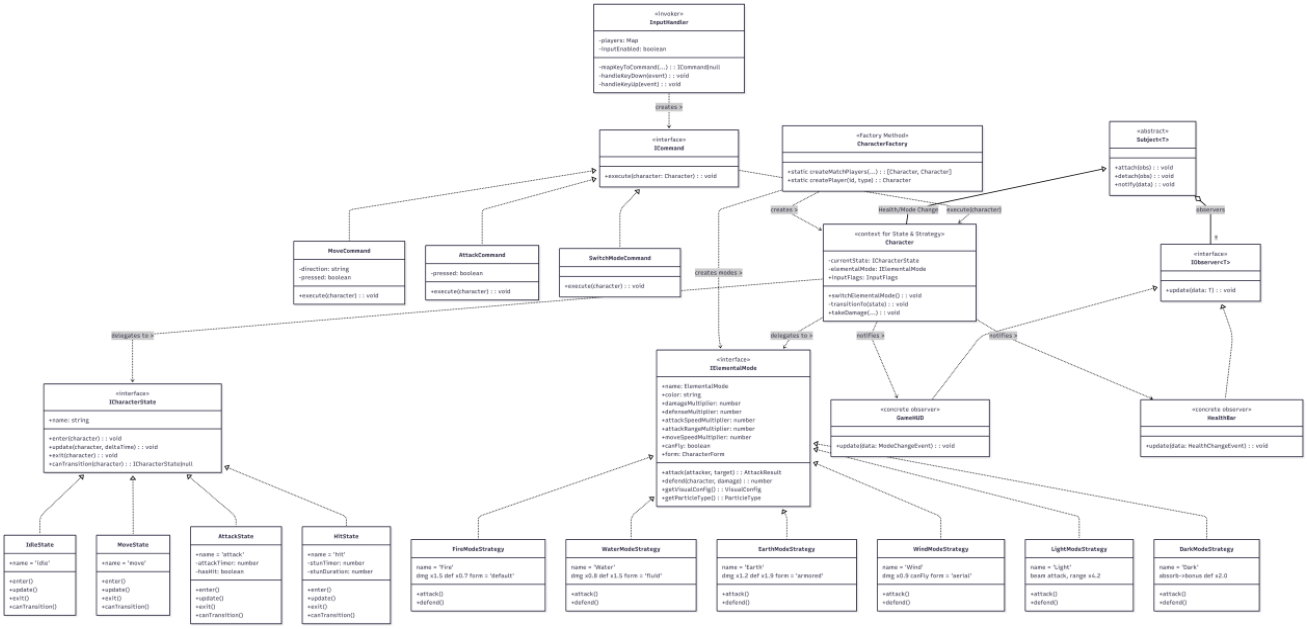
6.3.1 UML Class Diagram A – Core Architecture

The diagram below presents the structural backbone of Color Clash. GameEngine (Singleton) acts as the central orchestrator, owning InputHandler, ParticleSystem, and GameHUD as subsystems. Character extends Subject<T>, enabling it to publish HealthChangeEvent and ModeChangeEvent notifications. Both Hitbox and Particle implement IPoolable for pool-compatibility.



6.3.2 UML Class Diagram B – Gameplay & Design Patterns

Diagram B isolates the design-pattern relationships. Command decouples keyboard events from character actions. State governs per-frame behaviour. Strategy enables hot-swapping of combat logic. Observer propagates game events to UI.



6.3.3 Class / Interface Responsibility Summary

Class / Interface	Pattern Role	Responsibility	Key Public API
GameEngine	Singleton	Central game-loop orchestrator; owns all subsystems	getInstance(), init(), start(), stop(), restart(), togglePause()
Character	Context (State + Strategy) + Subject	Main player entity; delegates behaviour to state/strategy and publishes events	update(), render(), takeDamage(), switchElementalMode(), checkAttackHit(), reset()
Hitbox	Value Object / IPoolable	AABB collision box for attack detection; poolable	init(), intersects(), getRect(), reset()
Particle	IPoolable	Single visual-effect particle with physics and typed rendering	init(), update(), render(), reset()
CharacterFactory	Factory Method	Constructs and wires two Character instances	createPlayer(), createMatchPlayers()
InputHandler	Command Dispatcher	Maps keyboard events to ICommand objects and executes them	registerPlayer(), destroy()

Class / Interface	Pattern Role	Responsibility	Key Public API
CollisionSystem	Utility (static)	Stateless AABB detection and resolution	checkAABB(), resolveCharacterCollision(), getCollisionPoint()
ParticleSystem	Pool Consumer	Spawns and manages particles via ObjectPool<Particle>	spawnHitEffect(), update(), render(), clear()
GameHUD	IObserver<ModeChangeEvent>	Composite UI; owns HealthBars and observes mode-change events	subscribeToCharacter(), tick(), render(), reset()
HealthBar	IObserver<HealthChangeEvent>	Animated health bar that receives health events from Character	update(), tick(), render(), reset()
ICommand / MoveCommand / AttackCommand / SwitchModeCommand	Command family	Decouples keyboard input from gameplay actions	execute(character)
ICharacterState / IdleState / MoveState / AttackState / HitState	State family	Controls character state transitions and state-specific updates	enter(), update(), exit(), canTransition()
IElementalMode / FireModeStrategy / WaterModeStrategy	Strategy family	Encapsulates combat-attribute variants and mode-specific behaviour	attack(), defend(), getVisualConfig(), getParticleType()
Subject<T> / IObserver<T>	Observer family	Generic observable and observer callbacks for UI event propagation	attach(), detach(), notify(), update(data: T)
ObjectPool<T>	Object Pool	Pre-allocates reusable poolable objects to prevent GC spikes	acquire(), release(), releaseAll(), getActiveObjects()
GameConfig	Constants Module	Centralised numeric and structural game constants	Exported const values

6.4 Interfaces Between Components

The following tables formally document the major component interaction points. Each entry lists the calling component, receiving component, transferred data, return value, and observable side effects.

6.4.1 Core Engine Interfaces

Interaction	Caller	Callee	Inputs / Returns / Side Effects
GameEngine.getInstance(canvas)	main.ts::initGame()	GameEngine	Input: HTMLCanvasElement (first call). Return: GameEngine singleton. Side effects: constructs engine, subsystems, and canvas size.
GameEngine.init()	main.ts::initGame()	GameEngine	Creates players via CharacterFactory, registers them with InputHandler, and subscribes HUD observers.
GameEngine.start()	main.ts::initGame()	GameEngine	Sets isRunning=true and schedules requestAnimationFrame.
GameEngine.update(deltaTime)	GameEngine::gameLoop()	GameEngine	Updates players, resolves collisions, ticks particles and HUD, spawns hit particles, and checks win state.
GameEngine.render()	GameEngine::gameLoop()	GameEngine	Draws background, particles, characters, HUD, and victory overlay.
checkAndSpawnHitParticles()	GameEngine::update()	CollisionSystem / ParticleSystem	Checks active hitboxes via getCollisionPoint() and calls spawnHitEffect() on overlap.
GameEngine.restart()	showVictory() / keydown R	GameEngine	Resets positions, clears particles, resets HUD, and restores game state.
GameEngine.togglePause()	main.ts keydown P	GameEngine	Toggles isPaused; paused game skips update() but still renders.

6.4.2 Gameplay / Combat Interfaces

Interaction	Caller	Callee	Inputs / Returns / Side Effects
InputHandler::handleKeyDown()	Browser keydown event	InputHandler	Calls mapKeyToCommand() and then command.execute(character).
InputHandler::mapKeyToCommand()	InputHandler::handleKeyDown/Up	InputHandler internal	Constructs MoveCommand, AttackCommand, or SwitchModeCommand from keyCode, key bindings, pressed flag, and playerId.
MoveCommand / AttackCommand / SwitchModeCommand.execute()	InputHandler	Character	Updates directional input flags, attack flags, or triggers switchElementalMode().
Character.update(deltaTime)	GameEngine::update()	Character -> ICharacterState	Ticks cooldowns, delegates to currentState.update(), and evaluates canTransition().
ICharacterState.update() / canTransition()	Character.update()	Concrete State	Mutates velocity and position, applies gravity, checks boundaries, and returns next state when appropriate.
AttackState.enter(character)	Character::transitionTo()	AttackState	Sets isAttacking=true, starts cooldown, and creates attack hitbox.
Character.checkAttackHit()	AttackState.update()	Character / IElemental Mode	Tests hitbox intersection, calls elementalMode.attack(), and applies opponent.takeDamage().
IElementalMode.attack() / defend()	Character combat flow	Concrete strategy	Computes damage, defense reduction, and knockback-related attack result.
CollisionSystem.resolveCharacterCollision()	GameEngine::update()	CollisionSystem	Checks overlap and pushes characters

Interaction	Caller	Callee	Inputs / Returns / Side Effects
			apart by overlap distance.
CharacterFactory.createMatchPlayers()	GameEngine.init()	CharacterFactory	Creates two players with proper positions, bindings, and opponent references.

6.4.3 UI / Observer Interfaces

Interaction	Caller	Callee	Inputs / Returns / Side Effects
GameHUD.subscribeToCharacter(ch)	GameEngine.init()	GameHUD / Character	Registers HealthBar and GameHUD as observers on the character.
Subject.notify(data)	Character.takeDamage() / switchElementalMode()	Subject<T> -> IObserver<T> *	Broadcasts HealthChangeEvent or ModeChangeEvent to registered observers.
HealthBar.update(data)	Subject.notify()	HealthBar	Updates currentHealth and damageFlashAlpha from HealthChangeEvent.
GameHUD.update(data)	Subject.notify()	GameHUD	Updates playerModes map for mode indicator rendering.
GameHUD.tick(deltaTime) / HealthBar.tick(deltaTime)	GameEngine::update() / GameHUD.tick()	GameHUD / HealthBar	Advances animations and interpolated health display.
GameHUD.render(ctx) / HealthBar.render(ctx)	GameEngine::render() / GameHUD.render()	GameHUD / HealthBar	Draws health bars, mode indicators, title text, damage flash, and labels.
GameHUD.reset()	GameEngine.restart()	GameHUD	Resets bars, clears mode-switch effects, and restores playerModes to Fire.

6.4.4 Performance / Pooling Interfaces

Interaction	Caller	Callee	Inputs / Returns / Side Effects
new ObjectPool<Particle>(factory, size)	new ParticleSystem()	ObjectPool<Particle>	Pre-allocates particle objects through prewarm() using factory and pool size.
ObjectPool.acquire()	ParticleSystem::spawn()	ObjectPool<Particle>	Returns recycled or new Particle, sets active=true, resets it, and adds it to activeObjects.
Particle.init(...)	ParticleSystem::spawn()	Particle	Populates fields, randomizes rotation and rotation speed, and prepares lifetime parameters.
ObjectPool.release(obj)	ParticleSystem::update()	ObjectPool<Particle>	Deactivates expired particle, removes it from active set, and pushes it back to pool.
Particle.update(deltaTime)	ParticleSystem::update()	Particle	Advances motion,

Interaction	Caller	Callee	Inputs / Returns / Side Effects
			applies gravity and alpha/scale decay, and returns liveness state.
ObjectPool.releaseAll()	ParticleSystem.clear()	ObjectPool<Particle>	Moves all active objects back to the pool and clears activeObjects.
ParticleSystem.spawnHitEffect(pos, type)	GameEngine::checkAndSpawnHitParticles()	ParticleSystem	Spawns pooled hit particles for collision feedback.
ObjectPool.getActiveObjects()	ParticleSystem::update()/render()	ObjectPool<Particle>	Provides read-only view over active objects.

7. Data Design

7.1 Data Model

Data is managed through a central configuration and runtime class instances.

7.1.1 Core Runtime Data Structures

Data Structure	Purpose	Representative Fields
GameState	Stores the current execution state of the game session	isRunning, isPaused, currentMode, winner, roundState
PlayerState	Holds per-player runtime state	playerId, name, position, velocity, health, currentState, currentElementalMode
InputFlags	Represents current actionable input status	left, right, up, down, attack, switchMode
TournamentState	Tracks tournament registration and	participants, currentRound,

Data Structure	Purpose	Representative Fields
	progression	matches, winners, podium
HealthChangeEvent	Transfers combat damage results to UI observers	playerId, currentHealth, maxHealth, damage
ModeChangeEvent	Transfers elemental mode changes to UI observers	playerId, newMode

7.1.2 Data Ownership and Update Responsibility

- `GameEngine` owns the global game loop and coordinates top-level state transitions.
- `Character` instances own player-centric runtime data such as health, movement state, and active elemental mode.
- `InputHandler` updates command-triggering input flags.
- `GameHUD` and `HealthBar` consume observer events and do not own combat state.
- Tournament-related state is updated through tournament flow control and result progression logic.

7.2 Data Flow Diagrams

7.2.1 Runtime Data Flow Interpretation

The runtime data flow follows a predictable loop. Browser keyboard input is captured by the InputHandler, translated into command objects, and applied to the related Character. Character updates may produce combat outcomes, state transitions, and observer events. These events are then consumed by GameHUD and HealthBar, while GameEngine remains responsible for frame progression, collision checks, particle spawning, and round-level control.

8. Design Patterns

8.1 Applied Design Patterns

- Singleton: The GameEngine ensures a single orchestrator for the game loop.
- Strategy: IElementalMode allows dynamic switching between all combat attributes.
- State: ICharacterState manages transitions between Idle, Move, Attack, and Hit.
- Command: ICommand decouples raw keyboard events from the Character logic.
- Observer: Decouples the UI (GameHUD) from the core game logic, updating health bars via notifications.
- Factory: CharacterFactory centralizes the creation and initialization of player characters.
- Object Pool: ParticleSystem reuses particles and hitboxes to prevent performance spikes.

8.2 Context and Justification

The State Pattern is critical to ensure a character cannot perform actions while in a "stunned" (Hit) state. The Strategy Pattern allows for runtime modification of combat stats without altering the core character class.

9. Implementation Notes

Movement uses a velocity-based system with Delta-Time (dt) integration. This ensures that character speed remains consistent regardless of whether the user has a 60Hz or 144Hz monitor.

10. User Interface Design

The UI consists of a HUD (Heads-Up Display) overlaying the Canvas. It includes dynamic health bars, mode indicators, and game-state overlays.

- Dynamic Health Bars (Observer-driven).
- Mode Indicators.
- Game State overlays (Start/Game Over).

10.1 Main UI Screens

Pre-Match Setup Screen

This screen allows players to select the game mode, enter player or participant names, and choose character archetypes before gameplay begins. In tournament mode, it also collects tournament size and participant configuration.

Active Match Screen

This is the primary gameplay screen rendered on HTML5 Canvas. It displays player positions, attacks, elemental effects, health bars, mode indicators, and the ongoing duel state.

Winner / Result Overlay

When a match ends, the game presents a result overlay indicating the winner and supporting the transition to the next appropriate game state.

Tournament Progression / Podium Screen

In tournament mode, the UI presents bracket progression and, at the end of the flow, shows final ranking information for the top competitors.

11. Performance Considerations

- Object Pooling: Hitboxes and particles are recycled from a pool to prevent "Garbage Collection spikes" during intense combat.
- Culling: Entities outside the viewport are not rendered to save GPU cycles.

11.1 Current Performance Validation Scope

The current implementation includes design-time performance measures such as object pooling and rendering-related optimizations. In addition, delta-time-based movement is used to preserve consistent gameplay behavior across different refresh-rate environments. However, the final project documentation does not claim a formal benchmark study for FPS or latency; instead, performance acceptability is supported by stable build execution, smoke validation, and manual gameplay verification in the tested environment.

12. Error Handling and Logging

- Exception Management: The GameEngine uses try-catch blocks during initialization to handle failures in canvas context acquisition.
- Logging: A custom Logger tracks state transitions and input mapping for debugging purposes.

12.1 Error Handling Scope

Error handling in the current version is focused on practical runtime safety rather than enterprise-grade recovery workflows. The main emphasis is safe initialization, controlled command processing, and predictable gameplay flow. This is consistent with the project's MVP scope as a standalone browser-based application. Logging is primarily intended to support debugging, state inspection, and developer-side validation of input and gameplay transitions.

13. Deployment and Installation Design

- Environment: Node.js v18+.
- Installation: npm install.
- Execution: npm run dev for local development; npm run build for production-ready static files.

14. Change Log

- v1.0: Initial core engine and physics implementation.
- v1.1: Integrated State, Command, and Strategy patterns.
- v1.2: Tournament flow, result overlays, and final podium presentation consolidated.
- v1.3: Interface documentation, traceability alignment, and final validation artifacts strengthened.
- v1.4: Final document consistency updates for demo and delivery readiness.

15. Future Work / Open Issues

- Audio Engine: Integration of Web Audio API for sound effects.
- Optional control-remapping support for future usability improvements.
- Formal benchmark instrumentation for FPS and latency reporting.
- Extended asynchronous asset-loading workflow for larger future content sets.

16. Task Assignment Matrix

Team Member	Contribution (Classes & Tasks)	Use Case Mapping
Mehmet Umur ÖZÜ	Core & Input Systems: Implemented the GameEngine singleton, InputHandler, IdleState, and the Command pattern infrastructure. Also integrated the Strategy pattern for dynamic elemental mode switching.	UC1: Start Game Battle; UC2: Execute Combat Actions; UC3: Switch Elemental Modes
Mustafa GÖZÜTOK	Entity & Physics: Developed the CharacterFactory, Character class, MoveState, Object Pooling, and the CollisionSystem for AABB detection. Contributed to character behavior adaptation for different elemental modes.	UC1: Start Game Battle; UC2: Execute Combat Actions; UC3: Switch Elemental Modes; UC4: Manage Tournament Flow
Mehmet Yasin TOSUN	Combat & UI: Implemented AttackState, HitState, Hitbox management, main.ts entry point, and authored the Software Design Document (SDD). Developed tournament mode UI and match flow handling.	UC2: Execute Combat Actions; UC3: Switch Elemental Modes; UC4: Tournament Mode Execution